Scikit-learn Documentation: Getting Started

Source: https://scikit-learn.org/stable/getting_started.html

Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.

Fitting and Predicting: Estimator Basics

Scikit-learn provides dozens of built-in machine learning algorithms and models, called estimators. Each estimator can be fitted to some data using its `fit` method.

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> clf = RandomForestClassifier(random_state=0)
>>> X = [[ 1,  2,  3],  # 2 samples, 3 features
...      [11, 12, 13]]
>>> y = [0, 1]  # classes of each sample
>>> clf.fit(X, y)
RandomForestClassifier(random_state=0)
```

The `fit` method generally accepts 2 inputs:

- The samples matrix (or design matrix) `X`. The size of `X` is typically `(n_samples, n_features)`, meaning samples are represented as rows and features are represented as columns.
- The target values `y`, which are real numbers for regression tasks, or integers for classification. For unsupervised learning tasks, `y` does not need to be specified.

Both `X` and `y` are usually expected to be NumPy arrays or equivalent array-like data types.

Once the estimator is fitted, it can be used for predicting target values of new data without needing to re-train:

```
>>> clf.predict(X)  # predict classes of the training data
array([0, 1])
>>> clf.predict([[4, 5, 6], [14, 15, 16]])  # predict classes of new data
array([0, 1])
```

Transformers and Pre-processors

Machine learning workflows are often composed of different parts. A typical pipeline consists of a pre-processing step that transforms or imputes the data, and a final predictor that predicts target values.

In scikit-learn, pre-processors and transformers follow the same API as estimator objects. Transformer objects don't have a `predict` method but rather a `transform` method that outputs a newly transformed sample matrix `X`:

```
>>> from sklearn.preprocessing import StandardScaler
>>> X = [[0, 15],
...      [1, -10]]
>>> StandardScaler().fit(X).transform(X)
array([[-1.,  1.],
       [ 1., -1.]])
```

Sometimes you want to apply different transformations to different features: `ColumnTransformer` is designed for these use-cases.

Pipelines: Chaining Pre-processors and Estimators

Transformers and estimators (predictors) can be combined together into a single unifying object: a `Pipeline`. The pipeline offers the same API as a regular estimator: it can be fitted and used for prediction with `fit` and `predict`. Using a pipeline also prevents data leakage, i.e. disclosing some testing data in your training data.

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score

>>> pipe = make_pipeline(
...     StandardScaler(),
...     LogisticRegression()
... )

>>> X, y = load_iris(return_X_y=True)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

>>> pipe.fit(X_train, y_train)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
>>> accuracy_score(pipe.predict(X_test), y_test)
0.97...
```

Model Evaluation

Fitting a model to some data does not entail that it will predict well on unseen data; this needs to be directly evaluated. `train_test_split` splits a dataset into train and test sets, but scikit-learn provides many other tools for model evaluation, in particular for cross-validation.

Below is a 5-fold cross-validation procedure using the `cross_validate` helper:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.model_selection import cross_validate

>>> X, y = make_regression(n_samples=1000, random_state=0)
>>> lr = LinearRegression()

>>> result = cross_validate(lr, X, y)  # defaults to 5-fold CV
>>> result['test_score']  # r_squared score is high because dataset is easy
array([1., 1., 1., 1., 1.])
```

Automatic Parameter Searches

All estimators have parameters (often called hyper-parameters) that can be tuned. The generalization power of an estimator often critically depends on a few parameters, e.g. a `RandomForestRegressor` has an `n_estimators` parameter that determines the number of trees in the forest, and a `max_depth` parameter that determines the maximum depth of each tree.

Scikit-learn provides tools to automatically find the best parameter combinations via cross-validation, e.g. `RandomizedSearchCV`:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.ensemble import RandomForestRegressor
>>> from sklearn.model_selection import RandomizedSearchCV, train_test_split
```

```
>>> from scipy.stats import randint

>>> X, y = make_regression(n_samples=20640, n_features=8, noise=0.1, random_state=0)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

>>> param_distributions = {'n_estimators': randint(1, 5),
...                        'max_depth': randint(5, 10)}

>>> search = RandomizedSearchCV(estimator=RandomForestRegressor(random_state=0),
...                             n_iter=5,
...                             param_distributions=param_distributions,
...                             random_state=0)
>>> search.fit(X_train, y_train)
>>> search.best_params_
{'max_depth': 9, 'n_estimators': 4}

>>> search.score(X_test, y_test)
0.84...
```

Note: In practice, you almost always want to search over a pipeline instead of a single estimator. Applying a pre-processing step to the whole dataset without a pipeline, then performing cross-validation, breaks the fundamental assumption of independence between training and testing data — this leads to over-estimating the generalization power of the estimator. Using a pipeline for cross-validation and searching keeps you from this common pitfall.

Next Steps

This guide covers estimator fitting and predicting, pre-processing steps, pipelines, cross-validation tools, and automatic hyper-parameter searches — an overview of some of the main features of scikit-learn.